Design write-up — GitHub org security & access-hygiene scanner

Problem

Scan every repository in a GitHub organization and produce two things a security lead can act on in minutes: (1) a consolidated **security-advisory report** (Dependabot alerts across all repos, by severity, state, and age) and (2) a **data-driven access review** that flags members whose current contribution no longer justifies their access. Access removal is always a **suggestion for human review — never an automatic action.**

Contribution scoring — the model and why

For each `(member, repo)` inside a lookback window (default **180 days**) I take four activity counts and combine them:

```
raw_score   = 1·commits + 2·prs_opened + 3·prs_merged + 2·reviews
score       = raw_score · recency_factor          # recency_factor = 0.5^(days_idle / 90)
relative(%) = 100 · score / (top *human* score on that repo)
```

Why these choices — each is a deliberate, defensible decision:

- **PRs and reviews are weighted above raw commits.** A merged PR is completed, integrated, usually-reviewed work; a review is a maintainership signal. Ranking by commit count alone punishes exactly the senior/admin/reviewer accounts we *least* want to deprovision. Weighting merges and reviews above raw commits protects them.
- **Exponential recency decay (90-day half-life).** The real question is "does this person *still* need access?", not "were they ever active?". Decay makes a contribution 6 months old worth a quarter of the same contribution today.
- **Relative score is measured against the top *human* contributor.** Bots are excluded from the baseline: a repo whose commits are dominated by a CI/bump bot must not deflate every human's relative score and trigger spurious flags. If a repo has no human activity we fall back to the overall max so the math stays defined.
- **A removal suggestion requires three conditions at once** — stale (no activity for > 180 days), low relative (< 15% of the top human), *and* low absolute (score < 3.0). Requiring all three means we neither punish everyone on a busy repo nor keep everyone on a dormant one; a recently-active newcomer with a small footprint is given the benefit of the doubt.

Priority = blast radius, not staleness. Every suggestion is stale by construction, so recency can't rank them. What a security lead triages on is *how much damage the idle grant could do*: `admin`/`maintain`, or an **outside** collaborator with `write`, are **high**; internal `write` is **medium**; `triage`/`read`/unknown is **low**. (An earlier draft ranked by "very stale", which was redundant — every flagged grant is already stale — and left the *low* tier unreachable. The blast-radius model fixes both and matches how humans prioritise cleanup.)

Assumptions & edge cases (all handled explicitly)

- **Owners are never suggested for removal.** Resolved via the org members/admin API. If that list can't be read (e.g. scanning a public org you don't administer), the tool **says so in a warning** and asks you to verify — it does not silently guess.
- **Bots / service accounts** (GitHub type `Bot`, `[bot]` suffix, or a known-bot allowlist) are excluded from suggestions and surfaced separately — revoking a service account is an operational change, not access hygiene.

- **Sole active maintainer is protected.** A repo with exactly one active human is flagged as a bus-factor risk, and that person is never suggested for removal.
- **Empty repos** are skipped (nothing to judge). **Brand-new repos** (< 30 days) are flagged and have suggestions suppressed — too little history to be fair.
- **Archived** and **forked** repos are labelled; archived repos still surface stale access (it returns on unarchive) but the evidence says so.
- **Low-commit reviewers/admins are not penalised** — the review weight and the PR/merge weighting keep them scored appropriately (verified in the sample: the security reviewer with 4 commits / 51 reviews is *not* flagged).
- **Outside collaborators** are classified separately (`affiliation=outside`) and get extra scrutiny in the evidence.
- **Teams vs. individuals.** Access reaching a member *via a team* is classified `access_via = team` (versus `direct`/`outside_collaborator`). The suggestion notes this because the fix differs — you remove a direct grant, but team-derived access is fixed at the team level.
- **Public-limited mode.** When collaborator/security APIs are gated, the tool transparently falls back to the contributor list, marks the run `public-limited`, caps priorities at `low` (no permission data to claim blast radius), and clearly labels any synthetic advisories as `MOCK`.

Idempotency, rate limits, and 100+ repo orgs

- **Idempotent by design.** The client keeps an on-disk **ETag cache**; re-runs send `If-None-Match`, and unchanged resources return `304` (which does **not** consume the primary rate limit). Given the same data, a re-run reproduces byte-identical output — demonstrated in the run log (identical SHA-256s).
- **Rate limits.** The client pauses when the primary budget drops below a floor (until reset), and honours secondary/abuse `Retry-After` with bounded retries.
- **Bounded work per repo.** Commit/PR pagination and per-PR review lookups are page-capped, so a single huge repo can't make a run unbounded. On a 100+ repo org the run is *serial and safe by default* (it will simply take longer and lean on the cache); the scaling section covers making it fast.

What I'd change for production at scale

- **Switch hot paths to the GraphQL API** to collapse many REST calls (commits + PRs + reviews + collaborators) into few queries — the biggest single win on a large org.
- **Parallelise per-repo** with a worker pool and shared, adaptive rate-limit backoff; persist the ETag cache in shared storage (S3/Redis) so scheduled runs and multiple workers share conditional-request savings.
- **Incremental scans** — only re-pull repos whose `pushed_at` changed since the last run.
- **Identity mapping** — join GitHub logins to the IdP (SSO/SCIM) so "access no longer justified" can incorporate *employment status* (a departed employee is a removal, not a suggestion) and team membership from the source of truth.
- **Real advisories at scale** via GitHub Advanced Security / the org-level Dependabot alerts endpoint, plus secret-scanning and code-scanning feeds.
- **History & trend** — persist each run to warehouse storage to show whether hygiene is improving, and to age-track advisories.
- **Secrets & least privilege** — tokens from a secrets manager, scoped read-only, short-lived; run as a GitHub App with fine-grained permissions rather than a PAT.
- **Delivery** — push the dashboard to an internal static host and alert via Slack/PagerDuty (the included n8n workflow is the first step toward this).